LAB CYCLE 1

1. Write a program to count
a. vowels, consonants, digits, and special characters in a string given as input.

CODE:

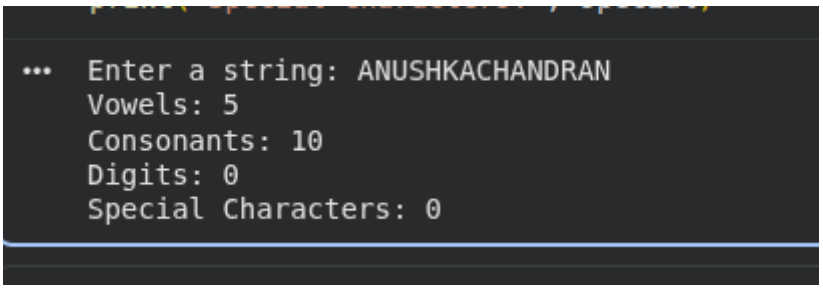
```
string = input("Enter a string: ")

vowels = 0
consonants = 0
digits = 0
special = 0

for ch in string:
    if ch.isalpha():
        if ch.lower() in "aeiou":
            vowels += 1
        else:
            consonants += 1
    elif ch.isdigit():
        digits += 1
    else:
        special += 1

print("Vowels:", vowels)
print("Consonants:", consonants)
print("Digits:", digits)
print("Special Characters:", special)
```

OUTPUT:



2. Write a program to find the largest and second largest element in a list

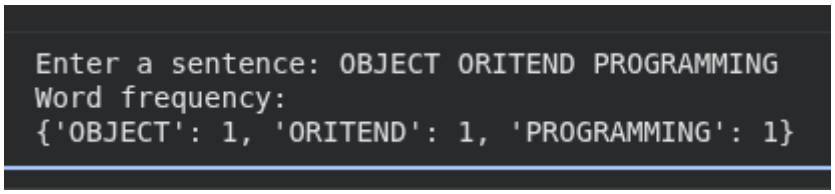
CODE:

```
numbers = [10, 25, 8, 45, 30, 45, 15]

unique_numbers = list(set(numbers))
unique_numbers.sort(reverse=True)

print("Largest element:", unique_numbers[0])
print("Second largest element:", unique_numbers[1])
```

OUTPUT:



3. Write a program to count word frequency using a dictionary.

CODE:

```
text = input("Enter a sentence: ")

words = text.split()
frequency = {}

for word in words:
    frequency[word] = frequency.get(word, 0) + 1

print("Word frequency:")
print(frequency)
```

OUTPUT:

```
Enter a sentence: WELCOME! PYTHON FILE HANDLING
Word frequency:
{'WELCOME!': 1, 'PYTHON': 1, 'FILE': 1, 'HANDLING': 1}
```

4. Write a program to sort dictionary by keys and the by values

CODE:

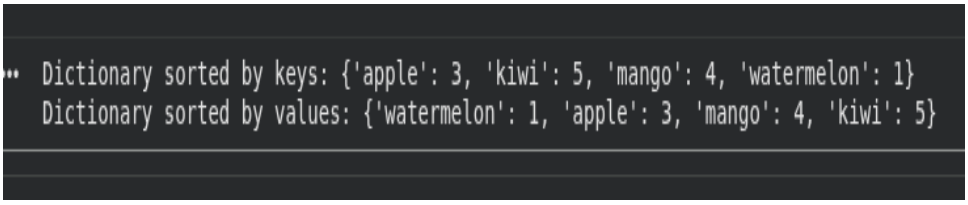
```
d = {'apple': 3, 'kiwi': 5, 'watermelon': 1, 'mango': 4}

# Sort by keys
sort_keys = dict(sorted(d.items()))

# Sort by values
sort_values = dict(sorted(d.items(), key=lambda item: item[1]))

print("Dictionary sorted by keys:", sort_keys)
print("Dictionary sorted by values:", sort_values)
```

OUTPUT:



5. Write a program using list comprehension to generate prime numbers

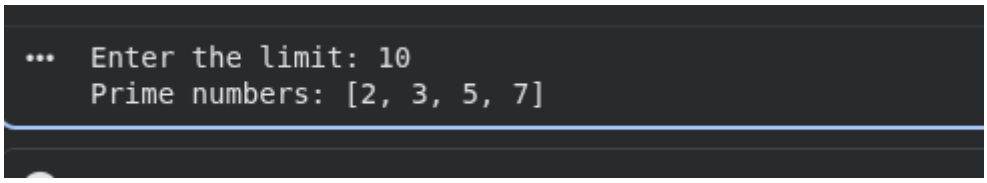
CODE:

```
n = int(input("Enter the limit: "))

primes = [num for num in range(2, n + 1)
          if all(num % i != 0 for i in range(2, int(num **
0.5) + 1))]

print("Prime numbers:", primes)
```

OUTPUT:



LAB CYCLE 2

6. Write a program to generate Fibonacci series using loops.

CODE:

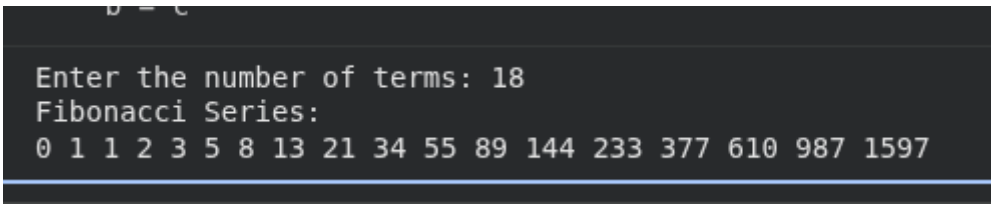
```
n = int(input("Enter the number of terms: "))

a = 0
b = 1

print("Fibonacci Series:")

for i in range(n):
    print(a, end=" ")
    c = a + b
    a = b
    b = c
```

OUTPUT:



7. Sort strings by length using lambda function.

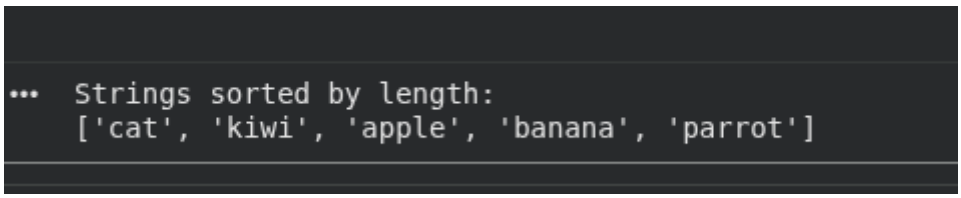
CODE:

```
strings = ["apple", "kiwi", "banana", "cat", "parrot"]

strings.sort(key=lambda x: len(x))

print("Strings sorted by length:")
print(strings)
```

OUTPUT:



8. Sort dictionary using multiple values and lambda function

CODE:

```
students = { "shamna": (20, 96), "pooja": (22, 91), "raghad": (23, 75) }

#Sort by age, then marks
sorted_students = sorted(students.items(), key=lambda x: (x[1][0], x[1][1]))
print(sorted_students)
```

OUTPUT:

```
[('shamna', (20, 96)), ('pooja', (22, 91)), ('raghad', (23, 75))]
```

LAB CYCLE 3

9. Write a program to demonstrate Inheritance and polymorphism.

CODE:

```
# a) Complex Number

class ComplexNumber:
    real = 0
    img = 0

    def __init__(self, rel, ig):
        self.real = rel
        self.img = ig

    def display(self):
        print(f"The complex number = {self.real}+{self.img}i")

    def __add__(self, other):
        real = self.real + other.real
        img = self.img + other.img
        return ComplexNumber(real, img)

    def __mul__(self, other):
        real = self.real * other.real - self.img * other.img
        img = self.real * other.img + self.img * other.real
```

```
        return ComplexNumber(real, img)



# Main program
print("Sum of these:")

c1 = ComplexNumber(5, 3)
c2 = ComplexNumber(2, 3)

c1.display()
c2.display()

c3 = c1 + c2
c3.display()

print("Product of these:")

c3 = c1 * c2
c3.display()



# b) Inheritance

class Person:
    def __init__(self, name, department):
        self.name = name
        self.department = department
```

```
    def display(self):
        print("NAME:", self.name)
        print("DEPARTMENT:", self.department)



class Student(Person):
    def __init__(self, name, department, perc):
        super().__init__(name, department)
        self.perc = perc

    def findgrade(self):
        if self.perc >= 90:
            print("Grade: A")
        elif self.perc >= 80:
            print("Grade: B")
        elif self.perc >= 70:
            print("Grade: C")
        else:
            print("Grade: D")

    def display(self):
        print("NAME:", self.name)
        print("DEPARTMENT:", self.department)
        print("PERCENTAGE:", self.perc)



class Teacher(Person):
    def __init__(self, name, department, salary):
```

```
        super().__init__(name, department)
        self.salary = salary

    def teach(self):
        print(self.name, "has salary")
        print(self.salary)



# Main program
s1 = Student("ANUSHKA", "BCA", 78)
s2 = Student("ANUSH", "BBA", 57)

t1 = Teacher("SHAMNA", "BBA", 134000)
t2 = Teacher("KRISHNEDHU", "BCA", 120000)

s1.findgrade()
s1.display()

print()

s2.findgrade()
s2.display()

print("*" * 25)

t1.display()
t1.teach()
```

print()

t2.display()

t2.teach()

OUTPUT:

```
Sum of these:
The complex number = 5+3i
The complex number = 2+3i
The complex number = 7+6i
Product of these:
The complex number = 1+21i
Grade: C
NAME: ANUSHKA
DEPARTMENT: BCA
PERCENTAGE: 78
Grade: D
NAME: ANUSH
DEPARTMENT: BBA
PERCENTAGE: 57
**********
NAME: SHAMNA
DEPARTMENT: BBA
SHAMNA has salary
134000
NAME: KRISHNEDHU
DEPARTMENT: BCA
KRISHNEDHU has salary
120000
```

10. Write a program to read and write text files .Use exceptions.

CODE:

try:

Write to a text file with open("sample.txt", "w") as file: file.write("Hello, World! \n") file.write("Welcome to Python programming.")

```
print("Data written successfully.")

# Read from the text file
with open("sample.txt", "r") as file:
    content = file.read()

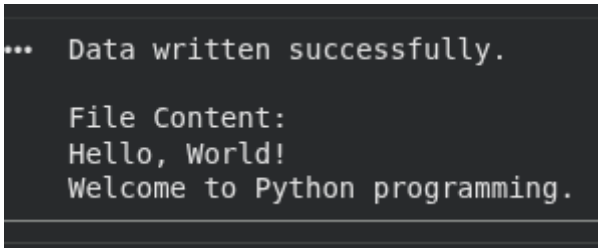
print("\nFile Content:")
print(content)
```

except FileNotFoundError: print("Error: File not found.")

except IOError: print("Error: Unable to read/write the file.")

except Exception as e: print("An error occurred:", e)

OUTPUT:



LAB CYCLE 4

11. Write a Python program using Pandas

a. to create and analyze a student dataset containing student name, marks and grade.

b. Use the following data: data = { "Name": ["Aman", "Diya", "Rahul", "Sneha", "Arjun"], "Marks": [78, 92, 85, 67, 74], "Grade": ["B", "A", "A", "C", "B"]}

c. Perform the following operations:

i. Create a Pandas DataFrame using the given dataset.

ii. Display the complete dataset.

iii. Display the first 3 rows of the dataset.

iv. Check whether the dataset contains missing values.

v. Create a histogram showing mark distribution.

vi. Find median and standard deviation of marks.

CODE:

```
import pandas as pd import matplotlib.pyplot as plt

#Given student data
data = { "Name": ["Aman", "Diya", "Rahul", "Sneha", "Arjun"], "Marks": [78, 92, 85, 67, 74], "Grade": ["B", "A", "A", "C", "B"] }

#i. Create a Pandas DataFrame
df = pd.DataFrame(data)

#ii. Display the complete dataset
print("Complete Dataset:") print(df)
```

#iii. Display the first 3 rows

print("\nFirst 3 Rows:") print(df.head(3))

#iv. Check for missing values

print("\nMissing Values:") print(df.isnull().sum())

#v. Create a histogram showing mark distribution

plt.hist(df["Marks"], bins=5,
edgecolor="black") plt.xlabel("Marks") plt.ylabel("Number of Students") plt.title("Distribution of Student Marks") plt.show()

#vi. Find median and standard deviation of marks

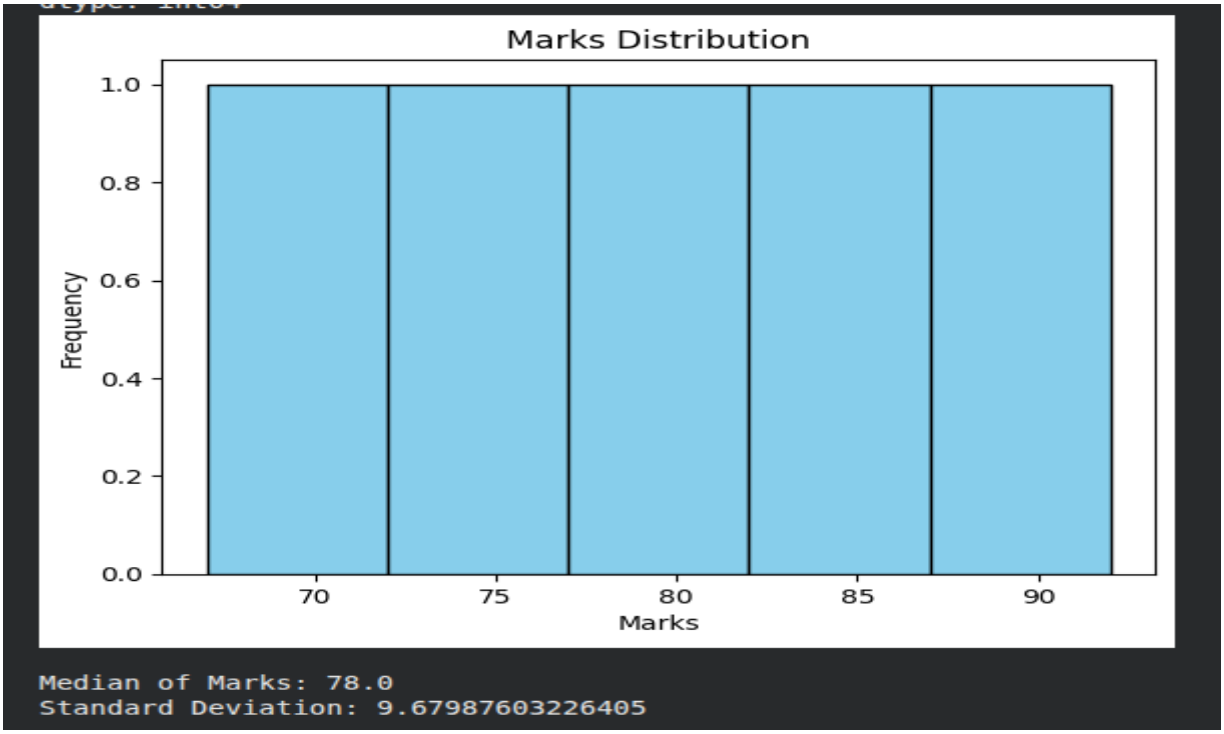
print("\nMedian of Marks:", df["Marks"].median()) print("Standard Deviation of Marks:", df["Marks"].std())

OUTPUT:

```
...  Complete Dataset:
        Name  Marks Grade
     0   Aman     78     B
     1   Diya     92     A
     2  Rahul     85     A
     3  Sneha     67     C
     4  Arjun     74     B

     First 3 Rows:
        Name  Marks Grade
     0   Aman     78     B
     1   Diya     92     A
     2  Rahul     85     A

     Missing Values:
     Name     0
     Marks    0
     Grade    0
     dtype: int64
```


Marks Distribution
Frequency
1.0
0.8
0.6
0.4
0.2
0.0
70
75
80
85
90
Marks
Median of Marks: 78.0
Standard Deviation: 9.67987603226405

12. Create dataframe from the given
a. data = {"Name": ["Aman", "Diya", "Rahul", "Sneha", Arjun"], "Department":
["CS", "IT", "CS", "ECE", "IT"], "Marks": [78, 92, 85, 67, 74], "Attendance":
[85, 90, 88, 75, 80], "Grade": ["B", "A", "A", "C", "B"]}.
b. Display Dataset Information
c. Display Statistical Summary
d. Display only student names and marks
e. Display students scoring more than 80 marks.
f. Sort students based on marks in descending order.
g. Add a new column named “Bonus” containing: 5 marks bonus for every student.

CODE:

```
import pandas as pd

#a. Create the DataFrame
data = { "Name": ["Aman", "Diya", "Rahul", "Sneha", "Arjun"], "Department": ["CS", "IT", "CS", "ECE", "IT"], "Marks": [78, 92, 85, 67, 74], "Attendance": [85, 90, 88, 75, 80], "Grade": ["B", "A", "A", "C", "B"] }
df = pd.DataFrame(data)

#b. Display Dataset Information
print("Dataset Information:") print(df.info())

#c. Display Statistical Summary
print("\nStatistical Summary:") print(df.describe())

#d. Display only student names and marks
print("\nStudent Names and Marks:") print(df["Name", "Marks"])

#e. Display students scoring more than 80 marks
print("\nStudents Scoring More Than 80:") print(df[df["Marks"] > 80])
```

#f. Sort students based on marks in descending order

print("\nStudents Sorted by Marks:") print(df.sort_values("Marks", ascending=False))

#g. Add Bonus column

df["Bonus"] = 5

print("\nDataFrame After Adding Bonus:") print(df)

OUTPUT:

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 5 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Name        5 non-null      object
 1   Department  5 non-null      object
 2   Marks       5 non-null      int64
 3   Attendance  5 non-null      int64
 4   Grade       5 non-null      object
dtypes: int64(2), object(3)
memory usage: 332.0+ bytes
None

Statistical Summary:
            Marks  Attendance
count   5.000000    5.000000
mean   79.200000   83.600000
std     9.679876    6.107373
min    67.000000   75.000000
25%    74.000000   80.000000
50%    78.000000   85.000000
75%    85.000000   88.000000
max    92.000000   90.000000
```

```
Student Names and Marks:
    Name  Marks
0   Aman     78
1   Diya     92
2  Rahul     85
3  Sneha     67
4  Arjun     74

Students Scoring More Than 80 Marks:
    Name Department  Marks  Attendance Grade
1   Diya         IT     92          90     A
2  Rahul         CS     85          88     A

Sorted by Marks (Descending):
    Name Department  Marks  Attendance Grade
1   Diya         IT     92          90     A
2  Rahul         CS     85          88     A
0   Aman         CS     78          85     B
4  Arjun         IT     74          80     B
3  Sneha        ECE     67          75     C

DataFrame After Adding Bonus:
    Name Department  Marks  Attendance Grade  Bonus
0   Aman         CS     78          85     B      5
1   Diya         IT     92          90     A      5
2  Rahul         CS     85          88     A      5
3  Sneha        ECE     67          75     C      5
4  Arjun         IT     74          80     B      5
```

13. Using the above dataframe answer the following questions.
a. Create a new column named “Total” by adding marks and bonus.
b. Find average marks department-wise.
c. Count the number of students in each grade category.
d. Calculate: mean, median,standard deviation of student marks.
e. Display students having maximum attendance.
f. Increase marks of all students by 5.

CODE:

```
import pandas as pd

#Given DataFrame
data = { "Name": ["Aman", "Diya", "Rahul", "Sneha", "Arjun"], "Department": ["CS", "IT", "CS", "ECE", "IT"], "Marks": [78, 92, 85, 67, 74], "Attendance": [85, 90, 88, 75, 80], "Grade": ["B", "A", "A", "C", "B"] }
df = pd.DataFrame(data)

#Add Bonus column
df["Bonus"] = 5

#a. Create Total column
df["Total"] = df["Marks"] + df["Bonus"] print("a. Total Marks:") print(df"Name", "Marks", "Bonus", "Total")

#b. Average marks department-wise
print("\nb. Average Marks Department-wise:") print(df.groupby("Department") ["Marks"].mean())

#c. Count students in each grade
print("\nc. Number of Students in Each Grade:") print(df["Grade"].value_counts())

#d. Mean, Median and Standard Deviation
print("\nd. Statistics of Marks:") print("Mean:", df["Marks"].mean()) print("Median:", df["Marks"].median()) print("Standard Deviation:", df["Marks"].std())
```

#e. Display student having maximum attendance

print("\ne. Student Having Maximum Attendance:") print(df[df["Attendance"] == df["Attendance"].max()])

#f. Increase marks of all students by 5

df["Marks"] = df["Marks"] + 5

print("\nf. Marks After Increasing by 5:") print(df["Name", "Marks"])

OUTPUT:

```
DataFrame with Total:
    Name Department  Marks  Attendance Grade  Bonus  Total
0   Aman         CS     78          85     B      5     83
1   Diya         IT     92          90     A      5     97
2  Rahul         CS     85          88     A      5     90
3  Sneha        ECE     67          75     C      5     72
4  Arjun         IT     74          80     B      5     79

Average Marks Department-wise:
Department
CS     81.5
ECE    67.0
IT     83.0
Name: Marks, dtype: float64

Grade-wise Student Count:
Grade
B    2
A    2
C    1
Name: count, dtype: int64

Mean Marks: 79.2
Median Marks: 78.0
Standard Deviation: 9.67987603226405

Student(s) with Maximum Attendance:
   Name Department  Marks  Attendance Grade  Bonus  Total
1  Diya         IT     92          90     A      5     97
```

```
Marks After Adding 5:
    Name Department  Marks  Attendance Grade  Bonus  Total
0   Aman         CS     83          85     B      5     83
1   Diya         IT     97          90     A      5     97
2  Rahul         CS     90          88     A      5     90
3  Sneha        ECE     72          75     C      5     72
4  Arjun         IT     79          80     B      5     79
```

14. Write a Python program using Pandas to clean and preprocess the following student dataset.
a. Perform the following operations:
i. Create a DataFrame containing student details.
ii. Identify missing values in the dataset.
iii. Replace missing values in the Marks column using the mean value.
iv. Remove duplicate rows from the dataset.
v. Convert student names into uppercase.
vi. Rename the column Marks to Score.
vii. Display the cleaned dataset.

CODE:

```
import pandas as pd

#i. Create a DataFrame containing student details
data = { "Name": ["Aman", "Diya", "Rahul", "Sneha", "Arjun", "Aman"], "Marks": [78, 92, None, 67, 74, 78], "Grade": ["B", "A", "A", "C", "B", "B"] }
df = pd.DataFrame(data)
print("Original Dataset:") print(df)

#ii. Identify missing values
print("\nMissing Values:") print(df.isnull().sum())

#iii. Replace missing values in Marks with mean
mean_marks = df["Marks"].mean() df["Marks"] = df["Marks"].fillna(mean_marks)

#iv. Remove duplicate rows
df = df.drop_duplicates()

#v. Convert student names into uppercase
df["Name"] = df["Name"].str.upper()

#vi. Rename Marks column to Score
df = df.rename(columns={"Marks": "Score"})
```

#vii. Display cleaned dataset

print("\nCleaned Dataset:") print(df)

OUTPUT:

```
Original Dataset:
    Name  Marks Grade
0   Aman   78.0     B
1   Diya   92.0     A
2  Rahul    NaN     A
3  Sneha   67.0     C
4  Arjun   74.0     B
5   Diya   92.0     A

Missing Values:
Name     0
Marks    1
Grade    0
dtype: int64

Cleaned Dataset:
    Name  Score Grade
0   AMAN   78.0     B
1   DIYA   92.0     A
2  RAHUL   80.6     A
3  SNEHA   67.0     C
4  ARJUN   74.0     B
```

LAB CYCLE 5

15. Plot simple mathematical functions.[y=3x^2+4x]

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100)
y = 3 * x**2 + 4 * x

plt.plot(x, y, label="y = 3x² + 4x")
plt.xlabel("x")
plt.ylabel("y")
plt.title("Plot of y = 3x² + 4x")
plt.grid(True)
plt.legend()
plt.show()
```

OUTPUT:

